



Complete Reference Notes

From Basics to Advanced — With Examples

Variables • Data Types • OOP • Modules • File I/O

Exceptions • Iterators • Generators • Decorators • More

■ Table of Contents

Chapter 1	— Introduction & Setup
Chapter 2	— Variables, Data Types & Operators
Chapter 3	— Control Flow
Chapter 4	— Functions
Chapter 5	— Data Structures
Chapter 6	— String Manipulation
Chapter 7	— Object-Oriented Programming
Chapter 8	— Modules & Packages
Chapter 9	— File Handling & I/O
Chapter 10	— Exception Handling
Chapter 11	— Iterators & Generators
Chapter 12	— Decorators
Chapter 13	— Comprehensions
Chapter 14	— Lambda, map, filter, zip
Chapter 15	— Regular Expressions
Chapter 16	— Date & Time
Chapter 17	— Math & Random
Chapter 18	— Collections Module
Chapter 19	— Functional Programming
Chapter 20	— Concurrency & Multithreading
Chapter 21	— Virtual Environments & pip
Chapter 22	— Best Practices & Tips

Introduction & Setup

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991. It emphasizes code readability and supports multiple programming paradigms.

Why Python?

- Simple, clean syntax — easy to read and write
- Vast standard library and third-party ecosystem
- Cross-platform: runs on Windows, macOS, Linux
- Used in web dev, data science, AI, automation, scripting and more

Installing Python

```
# Download from https://python.org
# Verify installation:
python --version      # e.g. Python 3.12.0
python3 --version     # on Linux/macOS
```

Running Python Code

```
# Interactive interpreter
python
>>> print('Hello, World!')
Hello, World!

# Run a script
python my_script.py
```

Your First Program

```
# hello.py
print('Hello, World!')
name = input('Enter your name: ')
print(f'Welcome, {name}!')
```

■ *Python uses indentation (4 spaces) instead of braces {} to define code blocks.*

Variables, Data Types & Operators

Variables

Variables are containers for storing data. Python is dynamically typed — you don't declare types explicitly.

```
x = 10          # int
y = 3.14        # float
name = 'Alice'  # str
is_active = True # bool
nothing = None  # NoneType

# Multiple assignment
a, b, c = 1, 2, 3
x = y = z = 0
```

Data Types

```
# Integers
age = 25
big = 1_000_000  # underscores for readability

# Floats
pi = 3.14159
sci = 1.5e3      # 1500.0

# Strings
s1 = 'hello'
s2 = "world"
s3 = '''multi
line'''

# Booleans
flag = True
done = False

# Check type
print(type(age))  #
print(type(pi))  #
```

Type Conversion

```
int('42')      # 42
float('3.14')   # 3.14
str(100)        # '100'
bool(0)         # False
bool(1)         # True
list('abc')     # ['a', 'b', 'c']
```

Arithmetic Operators

```
a, b = 10, 3
print(a + b)    # 13 (addition)
print(a - b)    # 7 (subtraction)
print(a * b)    # 30 (multiplication)
print(a / b)    # 3.333 (division)
print(a // b)   # 3 (floor division)
print(a % b)    # 1 (modulus)
print(a ** b)   # 1000 (exponentiation)
```

Comparison & Logical Operators

```
# Comparison
5 > 3      # True
5 == 5     # True
5 != 3     # True
5 >= 5     # True

# Logical
True and False # False
True or False  # True
not True       # False

# Identity & Membership
x = [1,2,3]
1 in x        # True
y = x
y is x        # True (same object)
```

Assignment Operators

```
x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 4    # x = 6
x **= 2   # x = 36
x %= 7    # x = 1
```

■ Use `type()` to inspect variable type and `isinstance(x, int)` to check if a variable is of a specific type.

Control Flow

if / elif / else

```
score = 85

if score >= 90:
    print('Grade: A')
elif score >= 80:
    print('Grade: B')
elif score >= 70:
    print('Grade: C')
else:
    print('Grade: F')
```

```
Grade: B
```

Ternary (One-Line if)

```
age = 20
status = 'adult' if age >= 18 else 'minor'
print(status)  # adult
```

for Loop

```
# Iterate over a list
fruits = ['apple', 'banana', 'cherry']
for fruit in fruits:
    print(fruit)

# range()
for i in range(5):          # 0,1,2,3,4
    print(i, end=' ')

for i in range(2, 10, 2):   # 2,4,6,8
    print(i, end=' ')

# enumerate
for idx, val in enumerate(fruits):
    print(f'{idx}: {val}')
```

while Loop

```
count = 0
while count < 5:
    print(count, end=' ')
    count += 1
# Output: 0 1 2 3 4

# Infinite loop with break
```

```
while True:
    user = input('Enter q to quit: ')
    if user == 'q':
        break
```

break, continue, pass

```
# break – exit loop
for i in range(10):
    if i == 5:
        break
    print(i, end=' ') # 0 1 2 3 4

# continue – skip iteration
for i in range(6):
    if i % 2 == 0:
        continue
    print(i, end=' ') # 1 3 5

# pass – placeholder
def todo():
    pass # implement later
```

for / while with else

```
for i in range(5):
    print(i, end=' ')
else:
    print('\nLoop finished!') # runs when loop completes normally

# else is skipped if break occurs
```

■ *The else clause on a loop executes only when the loop completes without hitting a break.*

Functions

Defining & Calling Functions

```
def greet(name):
    return f'Hello, {name}!'

print(greet('Alice'))    # Hello, Alice!
```

Default & Keyword Arguments

```
def power(base, exp=2):
    return base ** exp

print(power(3))           # 9   (exp defaults to 2)
print(power(2, 10))       # 1024
print(power(exp=3, base=4)) # 64 (keyword)
```

*args and **kwargs

```
# *args – variable positional arguments
def add(*nums):
    return sum(nums)

print(add(1, 2, 3, 4))    # 10

# **kwargs – variable keyword arguments
def show_info(**info):
    for key, val in info.items():
        print(f'{key}: {val}')

show_info(name='Bob', age=30, city='NY')
```

Return Multiple Values

```
def min_max(nums):
    return min(nums), max(nums)

lo, hi = min_max([3, 1, 7, 2])
print(lo, hi)    # 1 7
```

Scope: local & global

```
x = 10    # global

def modify():
    global x
    x = 99    # modifies global x

modify()
```



```
print(x) # 99
```

Nested Functions & Closures

```
def outer(msg):
    def inner():
        print(msg) # captures msg from outer
    return inner

say_hi = outer('Hi!')
say_hi() # Hi!
```

Recursive Functions

```
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5)) # 120

def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

print([fibonacci(i) for i in range(8)]) # [0,1,1,2,3,5,8,13]
```

■ *Avoid deep recursion (Python default limit is ~1000). Use iteration or `sys.setrecursionlimit()` carefully.*

Data Structures

Lists

```
nums = [1, 2, 3, 4, 5]

# Access
print(nums[0])      # 1
print(nums[-1])     # 5
print(nums[1:4])    # [2, 3, 4]

# Modify
nums.append(6)       # [1,2,3,4,5,6]
nums.insert(0, 0)    # [0,1,2,...]
nums.remove(3)       # removes first 3
popped = nums.pop()  # removes & returns last

# Useful methods
nums.sort()
nums.reverse()
print(len(nums))
print(nums.count(2))
print(nums.index(4))
```

Tuples

```
point = (10, 20)
x, y = point          # unpacking
print(x, y)           # 10 20

# Tuples are immutable
rgb = (255, 128, 0)
# rgb[0] = 100 # TypeError!

# Single-element tuple needs a comma
single = (42,)
```

Dictionaries

```
person = {'name': 'Alice', 'age': 30, 'city': 'NYC'}

# Access
print(person['name'])      # Alice
print(person.get('salary', 0)) # 0 (default)

# Modify
person['age'] = 31
person['email'] = 'a@b.com'
del person['city']
```

```
# Iterate
for key, val in person.items():
    print(f'{key} => {val}')

# Useful methods
print(person.keys())    # dict_keys([...])
print(person.values())
person.update({'age': 32, 'phone': '123'})
```

Sets

```
s1 = {1, 2, 3, 4}
s2 = {3, 4, 5, 6}

# Operations
print(s1 | s2)    # union: {1,2,3,4,5,6}
print(s1 & s2)    # intersection: {3,4}
print(s1 - s2)    # difference: {1,2}
print(s1 ^ s2)    # symmetric diff: {1,2,5,6}

# Methods
s1.add(10)
s1.discard(99)    # safe remove (no error if missing)
print(3 in s1)    # True
```

Nested Structures

```
students = [
    {'name': 'Alice', 'grades': [90, 85, 92]},
    {'name': 'Bob', 'grades': [78, 82, 88]},
]

for s in students:
    avg = sum(s['grades']) / len(s['grades'])
    print(f"{s['name']}: avg={avg:.1f}")
```

String Manipulation

String Basics

```
s = 'Hello, Python!'

print(len(s))          # 14
print(s.upper())       # HELLO, PYTHON!
print(s.lower())       # hello, python!
print(s.title())       # Hello, Python!
print(s.strip())       # removes whitespace
print(s[0:5])          # Hello
print(s[::-1])         # !nohtyP ,olleH (reverse)
```

String Methods

```
s = '  hello world  '
s.strip()              # 'hello world'
s.lstrip()             # 'hello world  '
s.rstrip()             # '  hello world'

s = 'hello,world,python'
s.split(',')           # ['hello','world','python']
','.join(['a','b','c']) # 'a,b,c'

'hello'.replace('l','r') # 'herro'
'hello'.find('ll')       # 2
'hello'.startswith('he') # True
'hello'.endswith('lo')   # True
'hello'.count('l')       # 2
```

f-Strings (Formatted String Literals)

```
name = 'Alice'
age = 30
pi = 3.14159

print(f'Name: {name}, Age: {age}')
print(f'Pi = {pi:.2f}')          # Pi = 3.14
print(f'{{10*5}}')              # 50
print(f'{{'hello':^20}}')        # centered in 20 chars
print(f'{{1000000:,.}}')         # 1,000,000
print(f'{{0.875:.1%}}')          # 87.5%
```

String Formatting (older styles)

```
# .format()
'{ } + { } = { }'.format(1, 2, 3) # '1 + 2 = 3'
'{name} is {age}'.format(name='Bob', age=25)
```

```
# % formatting
'%s is %d years old' % ('Alice', 30)
```

Multiline & Raw Strings

```
# Multiline
text = '''Line 1
Line 2
Line 3'''

# Raw string (ignores escape sequences)
path = r'C:\Users\Alice\Documents'
regex = r'\d+\.\d+'
```

■ *f-strings are the fastest and most readable way to format strings in modern Python (3.6+).*

Object-Oriented Programming (OOP)

Classes & Objects

```
class Dog:
    # Class attribute (shared by all instances)
    species = 'Canis lupus familiaris'

    # Constructor
    def __init__(self, name, age):
        self.name = name    # instance attribute
        self.age = age

    def bark(self):
        return f'{self.name} says: Woof!'

    def __str__(self):
        return f'Dog({self.name}, {self.age})'

# Create objects
d1 = Dog('Buddy', 3)
d2 = Dog('Max', 5)
print(d1.bark())    # Buddy says: Woof!
print(d2)           # Dog(Max, 5)
print(Dog.species)  # Canis lupus familiaris
```

Inheritance

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return 'Some sound'

class Cat(Animal):    # inherits Animal
    def speak(self):  # override
        return f'{self.name} says: Meow!'

class Dog(Animal):
    def speak(self):
        return f'{self.name} says: Woof!'

animals = [Cat('Whiskers'), Dog('Rex')]
for a in animals:
    print(a.speak())  # polymorphism!
```

Encapsulation & Properties

```

class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # private (name-mangled)

    @property
    def balance(self):           # getter
        return self.__balance

    @balance.setter
    def balance(self, amount):   # setter
        if amount < 0:
            raise ValueError('Balance cannot be negative')
        self.__balance = amount

acc = BankAccount(1000)
print(acc.balance)             # 1000
acc.balance = 2000             # uses setter
# acc.__balance                # AttributeError!

```

Class & Static Methods

```

class Counter:
    count = 0

    def __init__(self):
        Counter.count += 1

    @classmethod
    def get_count(cls):
        return cls.count

    @staticmethod
    def description():
        return 'I am a counter class'

Counter()
Counter()
print(Counter.get_count())    # 2
print(Counter.description())

```

Magic / Dunder Methods

```

class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x+other.x, self.y+other.y)

    def __repr__(self):
        return f'Vector({self.x}, {self.y})'

    def __len__(self):
        return 2

```

```
v1 = Vector(1, 2)
v2 = Vector(3, 4)
print(v1 + v2)    # Vector(4, 6)
print(len(v1))    # 2
```


Modules & Packages

Importing Modules

```
import math
print(math.sqrt(16))      # 4.0
print(math.pi)            # 3.14159...

from math import sqrt, pi
print(sqrt(25))           # 5.0

import os as operating_system
print(operating_system.getcwd())
```

Common Standard Library Modules

```
import os
os.getcwd()               # current directory
os.listdir('.')           # files in current dir
os.path.join('a', 'b.txt') # 'a/b.txt'
os.makedirs('folder', exist_ok=True)

import sys
print(sys.version)
print(sys.argv)           # command line args
sys.exit(0)
```

Creating Your Own Module

```
# mymath.py
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

PI = 3.14159

# main.py
from mymath import add, PI
print(add(2, 3)) # 5
print(PI)
```

Packages

```
# mypackage/
#   __init__.py
#   utils.py
#   models.py
```

```
# Import from package
from mypackage import utils
from mypackage.models import User
```

`__name__ == '__main__'`

```
# script.py
def main():
    print('Running main logic...')

if __name__ == '__main__':
    main()    # runs only when script is executed directly
             # NOT when imported as a module
```

■ *Always guard executable code with `if __name__ == '__main__':` to keep modules importable safely.*

File Handling & I/O

Reading Files

```
# Read entire file
with open('file.txt', 'r') as f:
    content = f.read()

# Read line by line
with open('file.txt') as f:
    for line in f:
        print(line.strip())

# Read all lines into a list
with open('file.txt') as f:
    lines = f.readlines()
```

Writing Files

```
# Write (overwrites)
with open('output.txt', 'w') as f:
    f.write('Hello, World!\n')
    f.writelines(['Line 1\n', 'Line 2\n'])

# Append
with open('output.txt', 'a') as f:
    f.write('Appended line\n')
```

File Modes

```
# 'r' - read (default)
# 'w' - write (creates/overwrites)
# 'a' - append
# 'x' - exclusive create (fails if exists)
# 'b' - binary mode (e.g., 'rb', 'wb')
# '+' - read & write (e.g., 'r+')

with open('image.png', 'rb') as f:
    data = f.read()
```

Working with JSON

```
import json

# Write JSON
data = {'name': 'Alice', 'age': 30, 'hobbies': ['coding', 'reading']}
with open('data.json', 'w') as f:
    json.dump(data, f, indent=4)

# Read JSON
```

```
with open('data.json') as f:
    loaded = json.load(f)
print(loaded['name']) # Alice

# Convert to/from string
json_str = json.dumps(data)
obj = json.loads(json_str)
```

Working with CSV

```
import csv

# Write CSV
rows = [['Name','Age'], ['Alice',30], ['Bob',25]]
with open('data.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(rows)

# Read CSV
with open('data.csv') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['Age'])
```

os.path utilities

```
import os

os.path.exists('file.txt')    # True/False
os.path.isfile('file.txt')    # is a file?
os.path.isdir('folder')      # is a directory?
os.path.basename('/a/b/c.txt') # 'c.txt'
os.path.dirname('/a/b/c.txt')  # '/a/b'
os.path.splitext('file.txt')  # ('file','.txt')
os.path.getsize('file.txt')    # size in bytes
```

Exception Handling

try / except / else / finally

```
try:
    x = int(input('Enter a number: '))
    result = 10 / x
except ValueError:
    print('That was not a number!')
except ZeroDivisionError:
    print('Cannot divide by zero!')
except Exception as e:
    print(f'Unexpected error: {e}')
else:
    print(f'Result: {result}')    # runs if no exception
finally:
    print('This always runs.')    # cleanup code
```

Common Built-in Exceptions

```
ValueError        # wrong value type
TypeError          # wrong type
IndexError         # list index out of range
KeyError           # dict key not found
AttributeError     # object has no such attribute
FileNotFoundError # file does not exist
ZeroDivisionError # division by zero
NameError          # variable not defined
StopIteration      # iterator exhausted
OverflowError      # numeric result too large
```

Raising Exceptions

```
def set_age(age):
    if not isinstance(age, int):
        raise TypeError('Age must be an integer')
    if age < 0 or age > 150:
        raise ValueError(f'Invalid age: {age}')
    return age

# Re-raise
try:
    set_age(-5)
except ValueError as e:
    print(f'Caught: {e}')
    raise    # re-raises the same exception
```

Custom Exceptions

```
class InsufficientFundsError(Exception):
    def __init__(self, amount, balance):
        self.amount = amount
        self.balance = balance
        super().__init__(
            f'Cannot withdraw {amount}. Balance: {balance}'
        )

def withdraw(balance, amount):
    if amount > balance:
        raise InsufficientFundsError(amount, balance)
    return balance - amount

try:
    withdraw(100, 200)
except InsufficientFundsError as e:
    print(e) # Cannot withdraw 200. Balance: 100
```

■ *Use specific exception types rather than bare 'except:' clauses to avoid masking unexpected errors.*

Iterators & Generators

Iterators

```
# Any object with __iter__ and __next__
nums = [1, 2, 3]
it = iter(nums)
print(next(it))    # 1
print(next(it))    # 2
print(next(it))    # 3
# next(it) now raises StopIteration

# Custom Iterator
class Countdown:
    def __init__(self, start):
        self.n = start

    def __iter__(self):
        return self

    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1

for n in Countdown(3):
    print(n)        # 3 2 1
```

Generators

```
# Generator function – uses yield
def count_up(n):
    i = 0
    while i < n:
        yield i
        i += 1

gen = count_up(5)
print(next(gen))    # 0
print(next(gen))    # 1

# Use in for loop
for val in count_up(3):
    print(val)       # 0 1 2
```

Generator Expressions

```
# Like a list comprehension but lazy
gen = (x**2 for x in range(10))
```

```
print(next(gen))    # 0
print(next(gen))    # 1

# Memory efficient – values computed on demand
total = sum(x**2 for x in range(1000000)) # no list created!
```

Infinite Generator

```
def natural_numbers():
    n = 1
    while True:
        yield n
        n += 1

gen = natural_numbers()
print([next(gen) for _ in range(5)]) # [1,2,3,4,5]
```

■ *Generators are memory-efficient for large data streams. They compute values lazily, one at a time.*

Decorators

What is a Decorator?

A decorator is a function that takes another function as input, adds some behavior, and returns a new function. It's syntactic sugar for higher-order functions.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print('Before call')
        result = func(*args, **kwargs)
        print('After call')
        return result
    return wrapper

@my_decorator
def say_hello(name):
    print(f'Hello, {name}!')

say_hello('Alice')
# Before call
# Hello, Alice!
# After call
```

Practical Decorators

```
import time
from functools import wraps

# Timer decorator
def timer(func):
    @wraps(func) # preserves docstring/name
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f'{func.__name__} took {end-start:.4f}s')
        return result
    return wrapper

@timer
def slow_func():
    time.sleep(0.5)

slow_func() # slow_func took 0.5002s
```

Decorator with Arguments

```
def repeat(n):
    def decorator(func):
```

```
@wraps(func)
def wrapper(*args, **kwargs):
    for _ in range(n):
        func(*args, **kwargs)
    return wrapper
return decorator

@repeat(3)
def hi():
    print('Hello!')

hi() # prints Hello! three times
```

Stacking Decorators

```
@decorator_a
@decorator_b
def my_func():
    pass

# Equivalent to:
my_func = decorator_a(decorator_b(my_func))
```

Comprehensions

List Comprehensions

```
# Basic
squares = [x**2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With condition (filter)
evens = [x for x in range(20) if x % 2 == 0]
# [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

# Transform + filter
names = ['alice', 'bob', 'charlie', 'dave']
long_names = [n.title() for n in names if len(n) > 3]
# ['Alice', 'Charlie', 'Dave']
```

Nested List Comprehensions

```
# Flatten a 2D list
matrix = [[1,2,3],[4,5,6],[7,8,9]]
flat = [val for row in matrix for val in row]
# [1, 2, 3, 4, 5, 6, 7, 8, 9]

# 2D grid
grid = [(r, c) for r in range(3) for c in range(3)]
# [(0,0),(0,1),(0,2),(1,0),..., (2,2)]
```

Dict Comprehensions

```
# Basic
squares = {x: x**2 for x in range(6)}
# {0:0, 1:1, 2:4, 3:9, 4:16, 5:25}

# Swap keys & values
original = {'a': 1, 'b': 2, 'c': 3}
swapped = {v: k for k, v in original.items()}
# {1:'a', 2:'b', 3:'c'}

# Filter
prices = {'apple': 1.2, 'grape': 3.5, 'banana': 0.8}
cheap = {k:v for k,v in prices.items() if v < 2}
# {'apple': 1.2, 'banana': 0.8}
```

Set Comprehensions

```
# Unique squares
nums = [1, -1, 2, -2, 3, 3]
unique_sq = {x**2 for x in nums}
# {1, 4, 9}
```

■ *Comprehensions are more Pythonic and often faster than equivalent for-loop constructs. But avoid making them too complex — readability matters.*

Lambda, map, filter, zip, reduce

Lambda Functions

```
# Anonymous one-liner function
square = lambda x: x ** 2
add = lambda a, b: a + b

print(square(5))    # 25
print(add(3, 4))    # 7

# Common use: sorting with key
words = ['banana', 'apple', 'cherry', 'date']
words.sort(key=lambda w: len(w))
print(words) # ['date', 'apple', 'banana', 'cherry']
```

map()

```
nums = [1, 2, 3, 4, 5]

# Apply function to each element
squares = list(map(lambda x: x**2, nums))
print(squares) # [1, 4, 9, 16, 25]

# With multiple iterables
a = [1, 2, 3]
b = [4, 5, 6]
result = list(map(lambda x, y: x + y, a, b))
print(result) # [5, 7, 9]
```

filter()

```
nums = range(1, 11)

# Keep elements where function returns True
evens = list(filter(lambda x: x % 2 == 0, nums))
print(evens) # [2, 4, 6, 8, 10]

words = ['hi', 'hello', 'hey', 'world']
h_words = list(filter(lambda w: w.startswith('h'), words))
print(h_words) # ['hi', 'hello', 'hey']
```

zip()

```
names = ['Alice', 'Bob', 'Carol']
scores = [95, 87, 92]

pairs = list(zip(names, scores))
print(pairs) # [('Alice', 95), ('Bob', 87), ('Carol', 92)]
```

```
# Iterate together
for name, score in zip(names, scores):
    print(f'{name}: {score}')

# Unzip
unzipped_names, unzipped_scores = zip(*pairs)
```

reduce()

```
from functools import reduce

nums = [1, 2, 3, 4, 5]

# Sum all
total = reduce(lambda acc, x: acc + x, nums)
print(total)    # 15

# Product
product = reduce(lambda acc, x: acc * x, nums)
print(product)  # 120
```

Regular Expressions

Basics

```
import re

# Search for pattern
match = re.search(r'\d+', 'I have 42 apples')
if match:
    print(match.group())    # '42'
    print(match.start())    # 7

# Find all occurrences
nums = re.findall(r'\d+', 'a1 b22 c333')
print(nums)    # ['1', '22', '333']

# Match from start of string
m = re.match(r'Hello', 'Hello, World!')
print(bool(m))    # True
```

Common Patterns

```
# \d - digit
# \w - word char (letter/digit/_ )
# \s - whitespace
# . - any char (except newline)
# ^ - start of string
# $ - end of string
# * - 0 or more
# + - 1 or more
# ? - 0 or 1
# {n} - exactly n times
# [abc] - char class
# (x|y) - x or y
# (group) - capture group

# Email validation (simplified)
pattern = r'^[\w.+-]+@[ \w-]+\.[a-zA-Z]{2,}$'
print(bool(re.match(pattern, 'user@example.com')))    # True
```

Substitution & Splitting

```
text = 'Hello    World    Python'

# Replace
clean = re.sub(r'\s+', ' ', text)
print(clean)    # 'Hello World Python'

# Split on multiple delimiters
parts = re.split(r'[;,|]+' , 'a,b;c|d')
```

```
print(parts)    # ['a', 'b', 'c', 'd']
```

■ *Always use raw strings (r'...') for regex patterns to avoid backslash confusion.*

Date & Time

```
from datetime import datetime, date, time, timedelta

# Current date & time
now = datetime.now()
print(now)                                # 2024-01-15 14:30:00.123456
print(now.year, now.month, now.day)
print(now.strftime('%Y-%m-%d %H:%M:%S'))

# Create specific date
d = date(2024, 6, 15)
print(d)                                  # 2024-06-15

# Timedelta
delta = timedelta(days=7)
next_week = now + delta
print(next_week.date())

# Difference between dates
d1 = date(2024, 1, 1)
d2 = date(2024, 12, 31)
diff = d2 - d1
print(diff.days)    # 365

# Parse string to datetime
dt = datetime.strptime('2024-06-15', '%Y-%m-%d')
print(dt)
```

Math & Random

```
import math

math.sqrt(16)      # 4.0
math.floor(3.7)    # 3
math.ceil(3.2)     # 4
math.factorial(5)  # 120
math.log(100, 10)  # 2.0
math.sin(math.pi/2) # 1.0
math.gcd(48, 18)   # 6
math.inf           # infinity
math.isnan(float('nan')) # True

import random

random.random()      # [0.0, 1.0)
random.randint(1, 10) # 1 to 10 inclusive
random.choice([1,2,3,4,5]) # pick one
random.choices([1,2,3], k=5) # pick 5 with replacement
random.sample([1,2,3,4,5], 3) # pick 3 without replacement
random.shuffle([1,2,3,4,5]) # in place
random.seed(42)        # reproducible results
```

Collections Module

```
from collections import Counter, defaultdict, OrderedDict, deque, namedtuple

# Counter – count element occurrences
words = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple']
c = Counter(words)
print(c)          # Counter({'apple':3, 'banana':2, 'cherry':1})
print(c.most_common(2)) # [('apple',3), ('banana',2)]

# defaultdict – dict with default value
dd = defaultdict(list)
dd['fruits'].append('apple')
dd['vegs'].append('carrot')
print(dd)  # defaultdict(<object>, {'fruits':['apple'],...})

# deque – efficient double-ended queue
dq = deque([1, 2, 3])
dq.appendleft(0)  # [0,1,2,3]
dq.append(4)      # [0,1,2,3,4]
dq.popleft()      # 0
dq.pop()          # 4

# namedtuple – tuple with named fields
Point = namedtuple('Point', ['x', 'y'])
p = Point(10, 20)
print(p.x, p.y)  # 10 20
```

Functional Programming

Pure Functions & Immutability

```
# Pure function: same input -> same output, no side effects
def add(a, b):
    return a + b    # pure

# Use tuple/frozenset for immutable collections
immutable = (1, 2, 3)
fs = frozenset([1, 2, 3])
```

functools

```
from functools import partial, lru_cache, reduce

# partial – fix some arguments
def power(base, exp):
    return base ** exp

square = partial(power, exp=2)
cube = partial(power, exp=3)
print(square(4))    # 16
print(cube(3))      # 27

# lru_cache – memoization
@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

print(fib(50))    # instant, cached results
```

Concurrency & Multithreading

Threading

```
import threading
import time

def worker(name, delay):
    print(f'{name} starting...')
    time.sleep(delay)
    print(f'{name} done!')

threads = [
    threading.Thread(target=worker, args=('A', 2)),
    threading.Thread(target=worker, args=('B', 1)),
]

for t in threads:
    t.start()

for t in threads:
    t.join()    # wait for all to finish

print('All threads done')
```

multiprocessing

```
from multiprocessing import Pool

def square(x):
    return x ** 2

with Pool(4) as p:
    results = p.map(square, range(10))
print(results)    # [0,1,4,9,16,25,36,49,64,81]
```

asyncio

```
import asyncio

async def fetch_data(name, delay):
    print(f'Fetching {name}...')
    await asyncio.sleep(delay)    # non-blocking
    print(f'{name} ready!')
    return f'data:{name}'

async def main():
    results = await asyncio.gather(
        fetch_data('A', 2),
        fetch_data('B', 1),
```

```
        fetch_data('C', 3),
    )
    print(results)

asyncio.run(main())
```

■ *Use threading for I/O-bound tasks, multiprocessing for CPU-bound tasks, and asyncio for high-concurrency async I/O.*

Virtual Environments & pip

Creating Virtual Environments

```
# Create
python -m venv myenv

# Activate
# Windows:
myenv\Scripts\activate

# macOS/Linux:
source myenv/bin/activate

# Deactivate
deactivate
```

pip — Package Manager

```
# Install a package
pip install requests
pip install numpy==1.24.0 # specific version

# Install from requirements file
pip install -r requirements.txt

# List installed packages
pip list
pip freeze > requirements.txt

# Upgrade / Uninstall
pip install --upgrade requests
pip uninstall requests

# Search
pip search pandas # (may be disabled)
```

requirements.txt

```
# requirements.txt
requests==2.31.0
numpy>=1.24.0
pandas
flask~=3.0
```

Best Practices & Tips

PEP 8 Style Guide

- Use 4 spaces for indentation (not tabs)
- Max line length: 79 characters
- Two blank lines between top-level functions/classes
- One blank line between methods inside a class
- snake_case for variables/functions, PascalCase for classes, UPPER_CASE for constants
- Use spaces around operators: `x = 1 + 2`

Useful Built-in Functions

```
# Type checks & conversions
isinstance(x, int)      # preferred over type(x) == int

# Itertools
sorted([3,1,2], reverse=True) # [3,2,1]
list(reversed([1,2,3]))       # [3,2,1]
list(enumerate(['a','b']))     # [(0,'a'),(1,'b')]

# all() / any()
all([True, True, False]) # False
any([False, False, True]) # True

# abs, round, divmod
abs(-5)                  # 5
round(3.14159, 2) # 3.14
divmod(10, 3)            # (3, 1)

# min, max with key
words = ['hi', 'hello', 'hey']
print(max(words, key=len)) # 'hello'
```

Pythonic Idioms

```
# Swap variables
a, b = b, a

# Check for None properly
if x is None: ...
if x is not None: ...

# Use enumerate instead of range(len(...))
for i, val in enumerate(mylist): ...

# Context managers for resources
```



```
with open('file') as f: ...

# Walrus operator := (Python 3.8+)
if n := len(data):
    print(f'Got {n} items')

# Unpacking
first, *rest = [1, 2, 3, 4, 5]
# first = 1, rest = [2, 3, 4, 5]
```

Debugging Tips

```
# Print debugging
print(f'{variable=}') # Python 3.8+ shows name & value

# Breakpoint (Python 3.7+)
breakpoint() # drops into pdb debugger

# Assertions
assert len(data) > 0, 'Data cannot be empty'

# Logging (better than print in production)
import logging
logging.basicConfig(level=logging.DEBUG)
logging.debug('Debug message')
logging.info('Info message')
logging.warning('Warning!')
logging.error('Error occurred')
```

■ **Congratulations!** You've covered all major Python concepts. Keep practicing, build projects, and explore libraries like NumPy, Pandas, Flask, FastAPI, and more!